Fluento / Luyenviet App Summary

Repo-based snapshot of the current app. Product name varies in-code: the repo folder is **fluento**, while the UI/config brand is **Luyenviet**.

What It Is	A React + Spring Boot writing-practice app focused on English sentence and paragraph work with AI-generated prompts, scoring, and feedback. The UI exposes sentence translation, paragraph writing, rankings, history, profile management, and an admin console.
Who It's For	Primary persona inferred from routes, copy, and content types: learners practicing English writing/translation, especially users working on daily writing, email, essay/story tasks, and IELTS Task 1/2.
What It Does	• Supports practice modes for single-sentence work and multi-sentence paragraph writing. • Lets users choose type, topic, tone, level, and sentence count before starting practice. • Generates paragraph/sentence content and AI feedback, then stores draft and submitted answers. • Shows recent practice, history, rankings, streaks, credits, and total practice time. • Allows Google sign-in plus local account/password flows. • Lets users add encrypted Gemini API keys, pick active models, and spend/reset credits. • Provides admin pages for users, paragraphs, paragraph sentences, user practices, roles, API keys, and credit transactions.
How It Works	• Frontend: React 19 + Vite + React Router + TanStack Query + Ant Design/Tailwind. It calls `/api` via Axios and refreshes JWT access tokens on 401. • Backend: Spring Boot 3.2 API under `/api`, with controllers for auth, users, paragraphs, paragraph sentences, user practices, API keys, rankings, and admin features. • Data layer: Spring Data JPA persists to MySQL; Flyway SQL migrations exist under `backend/src/main/resources/db/migration`. • AI path: paragraph generation, vocabulary hints, and sentence feedback go through a `ChatService` implemented by `GeminiChatService`, using user-stored encrypted API keys and per-call credit transactions. • Supporting services: Google OAuth clients, Cloudinary avatar uploads, JWT-based auth/security, IP rate limiting, OpenAPI docs, Sentry, and a daily credit reset cron job.
How To Run	• Minimal repo-backed path: run `docker-compose up --build` from the repo root. • This compose file starts MySQL, the Spring Boot backend, and the frontend containerized behind Nginx. • Open `http://localhost:1234` for the app; the frontend proxies `/api/` to the backend container. • Extra setup for full functionality: Google OAuth, Gemini/OpenAI-compatible AI config, Cloudinary, and JWT/encryption secrets are referenced in repo config. Exact local setup instructions are partially inconsistent across docs; use `docker-compose.yml`, `backend/src/main/resources/application-hub.yml`, `frontend/.env`, and `.env.production` as the current evidence sources.
Not Found In Repo	A single authoritative README for local setup, seeded demo credentials, and a clean product definition that resolves the Fluento vs. Luyenviet naming mismatch.